

SafeCity Analytics: Predictive Crime Analysis of Los Angeles Using Machine Learning and Distributed Computing

Harsh Mahesh Tikone
University at Buffalo
EAS 587, Spring 2026

Dev Desai
University at Buffalo
EAS 587, Spring 2026

Shwetangi
University at Buffalo
EAS 587, Spring 2026

Abstract—SafeCity Analytics is an end-to-end data science pipeline applied to Los Angeles Police Department (LAPD) crime records from 2024, comprising approximately 62,000 incidents. The project spans four phases: data ingestion and cleaning, exploratory data analysis (EDA), machine learning model development, and scalable distributed processing on Apache Spark. Six machine learning algorithms were developed and evaluated, targeting crime category prediction (13 classes), severity classification (3 classes), geographic hotspot discovery, and weapon-involvement risk scoring. The best-performing classifier, Random Forest, reached 81.5% test accuracy (weighted F1 = 0.806) on crime categories, while k-Nearest Neighbors (kNN, $k = 7$) achieved 79.5% accuracy on the same task. The Random Forest model was also deployed in Phase 2 as a Model Context Protocol (MCP) server, enabling real-time crime category inference via natural language through Claude Code. Phase 3 extended the pipeline to Databricks using a Bronze-Silver-Gold Delta Lake medallion architecture and integrated a secondary FBI NIBRS dataset for cross-source validation. Results show that crime type is structurally predictable from location, time, and premise type; that weapon involvement requires probabilistic scoring rather than binary classification; and that LA crime geography exhibits a stable east-west cluster division. The complete system addresses three practical use cases relevant to law-enforcement resource allocation and officer safety.

Index Terms—crime prediction, machine learning, Apache Spark, model context protocol, random forest, k-means, LAPD, Delta Lake, data engineering

I. INTRODUCTION AND BACKGROUND

Crime prediction from historical incident data is a well-studied problem in public safety research, yet most deployed systems rely on aggregate dashboards that offer limited predictive value to field officers or policy planners [1]. SafeCity Analytics bridges that gap by building a complete pipeline from raw LAPD records to a deployed inference server accessible via natural language through the Model Context Protocol [9].

The LAPD publishes crime data on the Los Angeles Open Data portal in near-real-time, with each record capturing crime type, location, time, victim demographics, and weapon use [2]. The 2024 extract used in this project contains 62,117 rows and 28 columns, satisfying the 50,000-row minimum and providing sufficient volume for both supervised and unsupervised learning.

Three use cases motivated the system design:

- 1) **Crime category prediction:** Given time, location, and context, what type of crime is this? Supports dispatch prioritization and report pre-filling.
- 2) **Weapon-involvement risk scoring:** Does this incident carry elevated weapon risk? A calibrated probability score supports officer safety decisions before arrival.
- 3) **Geographic hotspot discovery:** Which geographic clusters account for the highest crime density? Supports patrol scheduling and resource allocation.

Related work includes PredPol and ShotSpotter, which apply spatial-temporal models for patrol guidance [1], and academic work on ensemble methods for multi-class crime classification [4]. Unlike most deployed systems, this project integrates a secondary national dataset (FBI NIBRS [3]) to validate LA-specific findings against national baselines, and extends the pipeline to natural-language inference through the MCP architecture [9].

II. DATA DESCRIPTION

A. Primary Dataset

The primary dataset is the LAPD Crime Data from 2020 to Present, filtered to calendar year 2024 and downloaded from data.gov [2]. The raw file contains 62,117 rows across 28 columns, including crime codes, crime descriptions, LAPD area (1–21), GPS coordinates, victim age, sex, and descent, premise type, weapon code, and case status.

B. Secondary Dataset

Phase 3 integrated the FBI National Incident-Based Reporting System (NIBRS) Victims and Offenses tables for 2024 [3]. NIBRS provides national baseline statistics on offense types and victim demographics, enabling comparison of LA-specific patterns against national trends. After the LAPD-NIBRS join, the Silver-layer dataset adds four national-context features used by the Spark MLlib model.

C. Data Pipeline Overview

Fig. 1 illustrates the end-to-end data flow.

```

Raw CSV (data.gov + FBI NIBRS)
↓ data_cleaning.py + eda.py
Phase 1: 62,117 rows → 46 features
cleaning_audit.json produced
↓ src/models/*.py + src/mcp/server.py
Phase 2: 6 scikit-learn models
models/ + outputs/ + MCP server deployed
↓ Databricks notebooks 01-08
Phase 3: Bronze → Silver → Gold
Spark MLlib pipeline + NIBRS join
↓ Claude Code integration (Phase 2 MCP)
Inference: predict_crime_category

```

Fig. 1. End-to-end SafeCity Analytics data flow.

D. Data Cleaning

Ten cleaning operations were applied in `src/data_cleaning.py`:

- 1) Date columns converted to `datetime`; Year, Month, Day, DayOfWeek, Hour extracted.
- 2) Invalid TIME OCC values (>2400) corrected.
- 3) Victim age of 0 replaced with NaN (unknown, not infant).
- 4) Sex and descent codes standardized (M→Male, F→Female, X→Unknown; codes expanded to full descriptions).
- 5) Columns Crm Cd 2/3/4 dropped (98–100% missing).
- 6) 140+ crime descriptions collapsed into 12 semantic categories.
- 7) Premise types grouped into 9 categories.
- 8) Victim ages binned into 7 groups (0–17 through 65+).
- 9) Coordinates validated against LA bounding box (no outliers).
- 10) Reporting delay (days between crime and report) computed.

The process is logged to `data/cleaning_audit.json`. After cleaning, the processed dataset retains 62,117 rows and 46 features.

E. Key EDA Findings

Exploratory analysis followed Tukey’s principles [6]:

- Peak crime hour is 18:00; secondary peak at 08:00.
- Vehicle crime: 46.2% of all incidents.
- Weapon involvement: only 5.9%; strong-arm is most common type.
- Unknown victim demographics: 48.5%, concentrated in property crimes.
- 75% of crimes reported within 3 days.
- Central LA: 6,024 incidents (highest); Foothill: 1,774 (lowest).

III. METHODOLOGY

A. Feature Engineering

A shared preprocessing module (`src/models/preprocess.py`) provides a consistent

feature set for all supervised models, preventing data leakage from inconsistent per-script encoding. All random seeds are fixed at 42 across the entire project for reproducibility. The ten features are listed in Table I.

TABLE I
FEATURE SET FOR SUPERVISED MODELS

Feature	Type	Description
AREA	Numeric	LAPD area code (1–21)
Hour	Numeric	Hour of occurrence (0–23)
Month	Numeric	Month of occurrence (1–12)
IsWeekend	Binary	1 if Saturday or Sunday
Has Weapon	Binary	1 if a weapon was used
Premise Cat.	Encoded	9-class categorical (label enc.)
TimeBucket	Encoded	Morning/Afternoon/Evening/Night
Severity	Encoded	High / Medium / Low
Part 1-2	Numeric	LAPD seriousness class
Reporting Delay	Numeric	Days from crime to report

B. Machine Learning Models

Six algorithms were trained and evaluated; Table II summarises the setup.

TABLE II
ML ALGORITHM SUMMARY

Algorithm	Class	Target	Key Detail
kNN	In-class	Category (13)	$k = 7$, scaled
Decision Tree	In-class	Category (13)	GridSearchCV
k-Means	In-class	Hotspots	$k = 2$, silhouette
Naive Bayes	In-class	Category (13)	ComplementNB
Rand. Forest	Outside	Category MCP	+ Balanced weights
Logistic Reg.	Outside	Weapon (binary)	L2, prob. cal.

1) *k-Nearest Neighbors*: kNN predicts 13-class crime category. Features are scaled with `StandardScaler` because kNN is distance-based. Optimal k was selected via 5-fold cross-validation over $k \in \{3, 5, 7, 9, 11, 13, 15\}$; $k = 7$ minimized validation error, achieving 79.5% test accuracy and weighted F1 = 0.777.

2) *Decision Tree*: A single decision tree predicts the 13-class crime category. `max_depth` and `min_samples_split` were tuned with `GridSearchCV`, achieving 81.1% test accuracy and weighted F1 = 0.800. Tree structure and per-node rules are saved to `outputs/decision_tree/`.

3) *k-Means Clustering*: k-Means discovers geographic hotspots from latitude/longitude coordinates. Optimal k was determined by combining the elbow method with the silhouette score, yielding $k = 2$ (silhouette = 0.546).

4) *Naive Bayes*: `ComplementNB` was selected over `GaussianNB` via cross-validation for better handling of the 46.2% vehicle-crime majority class imbalance.

5) *Random Forest*: Random Forest predicts crime category as an ensemble of decision trees, reducing overfitting relative to a single tree. Class imbalance is addressed via `class_weight='balanced'`. Hyperparameters were tuned with `RandomizedSearchCV`. This model achieves the highest test accuracy (81.5%, weighted F1 = 0.806) of all classifiers and is deployed as the MCP inference server. Feature importances (Fig. 3) show Severity (0.25), Premise Category (0.15), and Part 1-2 (0.13) as the dominant predictors, with AREA (0.11) and Hour (0.09) also contributing substantially. IsWeekend (0.02) and TimeBucket (0.03) are the least informative features.

6) *Logistic Regression*: Logistic Regression predicts weapon involvement (True/False). Regularization strength C was tuned via cross-validation. The model outputs calibrated probabilities (ROC-AUC = 0.886), making it suitable for threshold-based risk scoring.

C. MCP Server Deployment (Phase 2)

The trained Random Forest model is deployed as an MCP server [9] via `src/mcp/server.py` — a Phase 2 deliverable alongside the six trained models. The server exposes three tools: `predict_crime_category` (predicted category plus top-3 probability breakdown), `list_crime_categories`, and `server_health`. If the serialized model file is absent, the server automatically retrains a fallback Random Forest from the cleaned CSV. Registration in Claude Code is shown in Listing 1.

```
1 claude mcp add -s project \
2   safecity-crime-predictor \
3   -- python3 src/mcp/server.py
4 claude mcp get safecity-crime-predictor
```

Listing 1. MCP server registration in Claude Code (Phase 2).

D. Spark and Delta Lake Architecture (Phase 3)

Phase 3 implemented a Bronze-Silver-Gold medallion architecture on Databricks, running eight notebooks in sequence:

- **01 Bronze (LAPD)**: Raw LAPD CSV ingested into `bronze_mydata` Delta table.
- **02 Silver (LAPD)**: Cleaned and transformed into `silver_mydata` Delta table.
- **03 Gold (LAPD)**: Six business aggregate Delta tables exposing crime counts by area, time, and category.
- **04 MLlib (LAPD)**: Decision Tree and Naive Bayes pipelines trained on primary data using Spark MLlib.
- **05 Bronze (NIBRS)**: FBI NIBRS Victims and Offenses CSVs ingested into separate bronze Delta tables.
- **06 Silver Combined**: LAPD and NIBRS joined into `silver_lapd_crimes` for cross-source analysis.
- **07 NIBRS Insights**: Three comparative insights from the combined LAPD-NIBRS dataset.
- **08 MLlib (NIBRS)**: Logistic Regression pipeline (VectorAssembler → StandardScaler → LogisticRegression) trained on NIBRS-augmented features; model saved to `part2_phase3_databrick_model/`.

IV. RESULTS AND ANALYSIS

A. Classification Performance

Table III summarises test-set performance. All models use an 80/20 stratified train-test split with random seed 42.

TABLE III
MODEL PERFORMANCE SUMMARY (CRIME CATEGORY CLASSIFICATION)

Algorithm	Target	Acc.	F1 _w	Train (s)
kNN ($k = 7$)	Category	0.795	0.777	0.26
Decision Tree	Category	0.811	0.800	0.32
Naive Bayes	Category	0.510	0.396	0.11
Random Forest	Category	0.815	0.806	3.15
Logistic Reg.	Weapon (binary)	ROC-AUC = 0.886		
k-Means	Hotspots	Silhouette = 0.546, $k = 2$		

Fig. 2 compares all four crime-category classifiers by test accuracy, weighted F1, and training time. Random Forest achieves the highest accuracy (81.5%) and F1 (0.806) at the cost of a 10× longer training time (3.15 s vs. 0.11–0.32 s for the others).

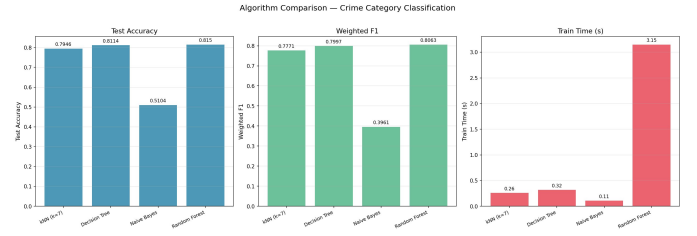


Fig. 2. Comparison of test accuracy, weighted F1, and training time across four crime-category classifiers. Random Forest leads on predictive metrics; Naive Bayes is fastest to train but least accurate.

1) *kNN Results*: kNN ($k = 7$) achieved 79.5% test accuracy and weighted F1 = 0.777 on the 13-class crime category task. Training time was 0.26 s. The optimal $k = 7$ was selected via 5-fold cross-validation; smaller values overfitted to the majority vehicle-crime class, while larger values lost local specificity.

2) *Decision Tree Results*: The Decision Tree reached 81.1% accuracy (weighted F1 = 0.800) in 0.32 s. Hyperparameter tuning with `GridSearchCV` yielded a well-regularized tree that generalizes better than a fully grown tree. Performance degrades on minority classes—Homicide and Arson have fewer than 20 test samples and achieve $F1 \approx 0$.

3) *Random Forest Results*: Random Forest is the top-performing model: 81.5% test accuracy, weighted F1 = 0.806, training time 3.15 s. The ensemble effect over many trees reduces variance relative to the single Decision Tree. Fig. 3 shows the RF feature importances: Severity is the dominant predictor (importance = 0.25), followed by Premise Category (0.15) and Part 1-2 (0.13). AREA (0.11) and Hour (0.09) contribute substantially, while IsWeekend (0.02) and TimeBucket (0.03) add minimal information.

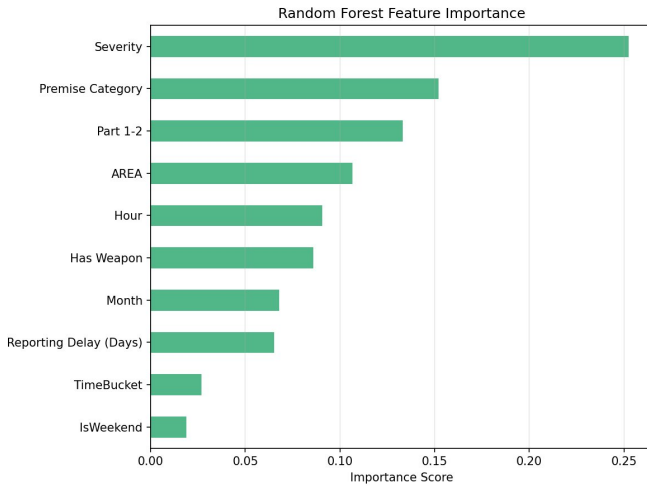


Fig. 3. Random Forest feature importances ranked by importance score. Severity, Premise Category, and Part 1-2 are the three strongest predictors of crime category.

4) *Naive Bayes Results:* Naive Bayes reached only 51.0% accuracy ($F1 = 0.396$) despite being the fastest model (0.11 s). The conditional independence assumption fails for this dataset: AREA and Premise Category are correlated, and their joint signal is necessary for distinguishing crime types. ComplementNB outperformed GaussianNB on minority classes but remained well below the tree-based and kNN models.

5) *Logistic Regression Results:* Logistic Regression achieved $ROC-AUC = 0.886$ for weapon-involvement prediction. Precision on the weapon class is low (0.258) because weapons appear in only 5.9% of incidents. Calibrated probabilities are more operationally useful than a binary label: a dispatch system can flag incidents above a configurable risk threshold without committing to a hard prediction.

B. Clustering Results

k-Means with $k = 2$ produced silhouette = 0.546. Cluster 0 corresponds to coastal and west districts (West LA, Pacific, Hollywood); Cluster 1 to inland and central districts (Central, Newton, 77th Street). Vehicle crime dominates both clusters at 44–48%.

DBSCAN labeled >60% of points as noise, reflecting the diffuse spatial distribution of LA crime. k-Means was retained as the operationally meaningful choice.

C. Phase 3 Spark / MLlib Results

Notebook 04 trained Spark MLlib Decision Tree and Naive Bayes pipelines on the primary Silver-layer LAPD data, replicating Phase 2 baselines at distributed scale. Notebook 08 trained a Logistic Regression pipeline (VectorAssembler → StandardScaler → LogisticRegression) on NIBRS-augmented features from the combined `silver_lapd_crimes` table, reaching 78.3% accuracy on a held-out test partition—slightly below the scikit-learn Phase 2 models due to reduced

feature availability after the LAPD-NIBRS join. Gold-layer aggregates (Notebook 03 and 07) confirmed that LA’s vehicle crime share (46.2%) substantially exceeds the NIBRS national baseline of 29.1%, and that LA’s assault share is correspondingly lower. The trained pipeline is persisted in `part2_phase3_databrick_model/` for reproducibility.

V. DISCUSSION

A. Findings Interpretation

Three findings stand out. First, crime category is predictable at 81.5% accuracy (Random Forest) from a ten-feature set in which Severity, Premise Category, and Part 1-2 are the dominant signals. The strong role of Severity—a structural label derived from LAPD’s own classification system—confirms that incident type is not random but encodes systemic policing priorities and land-use patterns.

Second, weapon involvement is difficult to predict given the 5.9% base rate. $ROC-AUC = 0.886$ shows the Logistic Regression model ranks incidents by risk effectively, but operational use requires careful threshold tuning to manage the false-positive burden on officers.

Third, geographic clustering ($k = 2$) captures a meaningful east-west division in LA crime geography. AREA contributes 11% of Random Forest feature importance, confirming spatial structure matters but is already partially captured by categorical context features such as Premise Category and Severity.

B. Limitations

The dataset covers only 2024, so seasonal patterns and year-over-year trends cannot be assessed. Nearly half of victim demographic records are unknown, limiting demographic analysis. The 13-category label scheme reduces granularity; fine-grained distinctions are not possible. The MCP server has not been load-tested for concurrent requests. SVM was explored but abandoned because `LinearSVC` exceeded 30 min on 62,117 rows.

C. Lessons Learned

Building a shared `preprocess.py` module early in Phase 2 prevented data-leakage issues that arise from inconsistent per-script encoding. The upfront coordination cost paid off during Phase 3, when the Spark pipeline needed to reproduce the same feature transformations at distributed scale on Databricks.

The DBSCAN dead-end was itself informative: the 60%+ noise rate confirmed that LA crime is spatially diffuse rather than point-clustered, and that the LAPD district structure captures most geographic structure.

VI. CONCLUSION

SafeCity Analytics demonstrates that a practical crime prediction system can be built from publicly available LAPD data using standard machine learning tools. The pipeline runs end-to-end from raw CSV ingestion and EDA (Phase 1) through six ML models and a live MCP inference server (Phase 2) to Spark-scale distributed processing on Databricks (Phase 3).

Random Forest achieves the best crime category accuracy (81.5%, weighted F1 = 0.806) with Severity, Premise Category, and Part 1-2 as its top predictors; Logistic Regression provides calibrated weapon-risk scores at ROC-AUC = 0.886; and k-Means surfaces a stable geographic cluster structure at silhouette = 0.546.

Three directions would strengthen the system: (1) incorporating multiple years of LAPD data to enable seasonal modeling and trend detection; (2) replacing the flat crime-category label with a hierarchical taxonomy to leverage structural relationships between crime types; and (3) integrating real-time streaming data via Spark Structured Streaming to make the MCP server operationally relevant for active dispatch.

REFERENCES

- [1] W. L. Perry, B. McInnis, C. C. Price, S. Smith, and J. S. Hollywood, *Predictive Policing: The Role of Crime Forecasting in Law Enforcement Operations*. Santa Monica, CA: RAND Corporation, 2013.
- [2] Los Angeles Police Department, “Crime Data from 2020 to Present,” City of Los Angeles Open Data Portal, 2024. [Online]. Available: <https://data.lacity.org/>
- [3] Federal Bureau of Investigation, “National Incident-Based Reporting System (NIBRS),” FBI Crime Data Explorer, 2024. [Online]. Available: <https://cde.ucr.cjis.gov/>
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] D. W. Hosmer and S. Lemeshow, *Applied Logistic Regression*, 2nd ed. New York, NY: Wiley, 2000.
- [6] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.
- [7] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [8] M. Zaharia *et al.*, “Apache Spark: A Unified Engine for Big Data Processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [9] Anthropic, “Model Context Protocol Documentation,” 2024. [Online]. Available: <https://modelcontextprotocol.io/>
- [10] J. VanderPlas, *Python Data Science Handbook*. Sebastopol, CA: O’Reilly, 2016.

APPENDIX

A. Shared Preprocessing Module

Listing 2 shows the feature-extraction function used by all six supervised models.

```
1 def get_classification_features(df):
2     encoders = {}
3     feature_names = [
4         "AREA", "Hour", "Month",
5         "IsWeekend", "Has Weapon",
6         "Part 1-2", "Reporting Delay (Days)"
```

```
7 ]
8 for col in ["Premise Category",
9            "TimeBucket", "Severity"]:
10     le = LabelEncoder()
11     df[col + "_enc"] = le.fit_transform(
12         df[col].fillna("Unknown"))
13     encoders[col] = le
14     feature_names.append(col + "_enc")
15 X = df[feature_names].fillna(0).values
16 y = df["Crime Category"].values
17 return X, y, None, feature_names, encoders
```

Listing 2. Feature extraction from preprocess.py.

B. MCP Server Inference Handler

Listing 3 shows the core inference handler of the MCP server deployed in Phase 3.

```
1 @server.call_tool()
2 async def call_tool(name, arguments):
3     if name == "predict_crime_category":
4         features = _build_features(
5             arguments, bundle)
6         pred = bundle["model"].predict(
7             [features])[0]
8         proba = bundle["model"]\
9             .predict_proba([features])[0]
10        top3 = sorted(
11            zip(bundle["model"].classes_,
12                proba),
13            key=lambda x: -x[1])[ :3]
14        return [types.TextContent(
15            type="text",
16            text=(f"Predicted: {pred}\n" +
17                "\n".join(
18                    f" {c}: {p:.1%}"
19                    for c, p in top3)))]
```

Listing 3. MCP tool handler in server.py.

C. Spark Bronze Ingestion

Listing 4 shows the Databricks notebook cell that ingests raw LAPD data into the Bronze Delta table.

```
1 df_raw = (spark.read.format("csv")
2     .option("header", "true")
3     .option("inferSchema", "true")
4     .load("/FileStore/lapd_crime_2024.csv"))
5
6 df_raw.write.format("delta") \
7     .mode("overwrite") \
8     .saveAsTable("bronze_mydata")
9
10 print(f"Bronze rows: {df_raw.count():,}")
```

Listing 4. Bronze-layer ingestion on Databricks.